

Cloud Platform Services – Technical Case Study

1) Project Overview

Cloud Platform Services (CPS) provides ongoing cloud operations support for a travel-technology platform hosted on AWS. The engagement focuses on maintaining production infrastructure and establishing a stable release workflow to deploy application releases across multiple environments—primarily production—while minimizing disruption during the client’s peak business hours.

The support started in late January 2025 and is currently in progress. The engagement is primarily support/operations-oriented, with continuous improvements to deployment automation and operational runbooks. A key driver is the client’s preference to avoid handling maintenance activities during peak hours.

2) Client & System Context

The client operates as a travel agency platform enabling customers to plan trips primarily in the USA, including booking flights, hotels, and rental cars, and managing end-to-end vacation planning. Historically focused on business travel, the company is expanding into personal travel. The domain is travel technology.

The system is small-to-medium in size and is active mainly during business hours (roughly 09:00–20:00 PST). While no formal SLA information was provided, the informal expectation is that downtime is not acceptable, including during maintenance windows.

The AWS footprint is concentrated in a single AWS account with four VPCs and a fleet of ~80 EC2 instances. The broader service inventory includes:

- ~25 Load Balancers
- ~30 Auto Scaling groups
- 2 Aurora clusters (9 DB instances total)
- 51 S3 buckets
- 14 Route 53 hosted zones
- 5 Amazon MQ brokers
- 7 MemoryDB clusters
- 8 DocumentDB clusters

- IAM: 18 user groups, 35 users, 202 roles, 138 policies, 3 identity providers
- 12 SNS topics (no SQS)
- ~50 Lambda functions and 7 Lambda applications

There is no EKS/Kubernetes footprint.

Environments (test, stage, prod plus additional intermediate/demo environments) are separated primarily via naming conventions, tags, DNS patterns, and load balancer configuration rather than strict account-level isolation.

3) Initial State

At the start of the engagement, deployment and operational processes were inconsistently defined and relied on a mixture of manual actions and partial automation. Specifically:

- Release workflow for the main application combined manual script execution, manual resource configuration, and pipeline execution.
- Documentation for release and rollback procedures was effectively absent.
- Environment boundaries and dependencies were not cleanly separated.

A recurring technical limitation was the lack of up-to-date documentation and constrained communication, which complicated changes, troubleshooting, and operational handover. Another major pain point was configuration drift: the client occasionally performed manual infrastructure changes that were not captured in infrastructure-as-code (IaC), causing mismatches between deployed resources and Terraform state.

4) Objectives

Key technical objectives of the engagement were:

- **Safer deployments** with **reduced manual steps**, particularly for production.
- Increased **automation** for deployments and infrastructure lifecycle tasks.
- Improved operational consistency and reduced deployment-related incidents.
- **Cost reduction**, where feasible within contractual time constraints.

Technical success was defined qualitatively rather than through explicit KPIs. The team aimed for a repeatable runbook-driven process, reduced incidents compared to the initial period, and a stable workflow that supports production changes outside the client's peak hours.

Maintenance activities within scope included DB maintenance, certificate renewals, infrastructure changes, application releases, and production database table changes.

5) Implemented Solution

CPS implemented a more standardized release and infrastructure workflow centered on CI/CD pipelines and infrastructure automation:

- **CI/CD and orchestration:** Jenkins pipelines were created to support both manual and automated triggers.
- **Infrastructure and configuration automation:**
 - **Terraform** was improved for provisioning and changing AWS infrastructure components.
- **Deployment strategy:**
 - **Blue/green deployments** were improved for **production** to reduce deployment risk and enable faster rollback.
- **Lifecycle management:**
 - Pipelines were designed not only to deploy applications, but also to create and configure AWS resources when needed, and to destroy resources in support of rollback or to revert unstable releases (where appropriate to the release pattern).

Cost optimization was limited to targeted storage cleanup actions due to contractual time constraints.

6) Execution & Effort

- **Timeline:** Ongoing support since late January 2025.
- **Team:** Three cloud engineers working part-time.
- **Collaboration model:**
 - Primary communication via Slack; meetings occur rarely.
 - Work requests are submitted via tickets; approvals happen via Slack confirmation and/or PR approvals.
- **Operating model vs peak hours:**
 - The CPS team operates from a different time zone with working hours largely outside the client's peak business window (09:00–20:00 PST), aligning maintenance and release activity to off-peak periods.
 - No on-call coverage is provided.

7) Challenges

Key technical challenges included:

- **Lack of documentation** at project start, especially for release and rollback workflows. This increased deployment risk and time-to-execution for changes.
- **Configuration drift and unmanaged changes**: occasional manual infrastructure changes by client-side personnel were not represented in Terraform, complicating IaC consistency and increasing the effort required to safely implement changes.

Trade-offs driven by constraints included:

- Leaving **IAM/access management** unchanged.
- Only limited progress on **cost optimization**, beyond some storage cleanup, due to time/contract limitations.

Mitigation steps focused on sustainability:

- The team documented processes and components that were created or improved during the engagement, building a growing set of operational documentation and repeatable runbooks.

8) Results & Learnings

Results (technical):

- A stable, repeatable deployment workflow was established, improving consistency across environments and reducing reliance on manual steps.
- Operational stability improved in practice through runbook-driven execution and reduced release incidents compared to the early phase of the engagement.

Learnings:

- The engagement highlighted the operational cost and delivery risk of ignoring platform best practices and recommendations—particularly around documentation, IaC discipline, and controlling manual changes.
- Establishing deployment patterns (e.g., blue/green) and codifying operational procedures early materially reduces risk for production changes, especially under “no downtime” expectations.

